University of Applied Sciences and Arts
Northwestern Switzerland
School of Computer Science
Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

Implementation of User-Centric Customization Interfaces and CSS-based Contrast Optimization

---

Author: **Florian Schnidrig**

Advisor: Prof. Dierk König

Profile: Computer Science

Date: Summer 2026 (TBD)

Contents

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion within modern web applications. It explored how semantic HTML serves as the backbone for accessibility, provided an overview of both common and specialized native elements, and clarified the roles of ARIA and WCAG. Furthermore, it offered foundational guidance on accessibility debugging, testing, and the practical use of screen readers.

Building upon that foundation, this project addresses essential aspects of accessibility that, while perhaps less technical in a traditional coding sense, are no less vital for an inclusive digital world. While our previous focus remained largely on the structural and technical "under the hood" mechanics, we have yet to explore the visual design landscape. These design choices are far from purely aesthetic; they play a decisive role in determining whether a web application is a welcoming space or a digital dead end for many users.

4. Theory

4.1. Testing Accessibility

4.2. CSS Media Queries

To begin, what exactly is a media query? At its core, a CSS media query allows developers to apply specific styles based on the characteristics of the device or environment displaying a web page. While most commonly associated with responsive design—adjusting layouts based on screen width—media queries are capable of much more than just reorganizing columns for mobile users.

The standard structure of a media query is as follows:

```
@media [not] media-type and (media-feature: value) and (media-feature:
value) {
  /* CSS rules to apply */
}
```

While we typically focus on the `screen` media type, styles can also be tailored for `print` (for those who still appreciate a physical copy) or all to cover every scenario.

Features like `max-width` and `max-height` are the industry workhorses for responsive websites. However, more specialized queries, such as `prefers-color-scheme`, have gained popularity by allowing developers to serve either a light or dark theme based on a user's system settings. Beyond aesthetics, several of these media features provide direct insight into a user's accessibility preferences and needs. Despite their power to create a more inclusive experience, they remain an underutilized tool in the developer's kit. [2]

The following media features are essential considerations for any web application aiming to be truly inclusive:

4.2.1. forces-colors

This media feature detects whether the user agent has enabled a forced colors mode, such as Windows High Contrast. When active, the operating system or browser takes the wheel, overwriting and restricting the colors defined in your stylesheet to ensure maximum readability. While developers can access these user-defined colors through the `system-color` CSS data type, this query should not be used to build a completely separate "high contrast" version of a site. Instead, it is best utilized for making subtle refinements where the automatic overrides might fall short.

For example, many modern buttons rely on a `box-shadow` to stand out from the background. Since forced colors mode typically sets `box-shadow` to `none` to reduce visual clutter, a button might lose its definition entirely. In this case, a developer can

use the `forced-colors` query to apply a solid border, ensuring the element remains visible and functional.

Beyond color overrides, certain style settings are automatically forced into values that prioritize clarity, such as:

- `box-shadow: none`
- `text-shadow: none`
- `color-scheme: light dark` (overridden by system preferences)

If a specific element absolutely requires its original color palette to remain functional, you can opt out of these overrides by setting `forced-color-adjust: none`. However, this should be used sparingly—after all, if a user has asked for high contrast, it's usually best to let them have it. [3], [4]

4.2.2. inverted-colors

When a user opts to invert their display colors, the results can be a bit unpredictable. What was once a subtle drop shadow may suddenly transform into a glowing highlight, inadvertently muddling the visual hierarchy and reducing readability. This media feature allows developers to detect this setting and make necessary adjustments to ensure the page's integrity remains intact.

By responding to this preference, you can "re-invert" specific elements—like images or videos—to ensure they don't look like photographic negatives, or refine text styles that have become strained under the new color palette. It is worth noting, however, that this feature is currently a bit of a specialist tool, as it is presently only supported by Safari. [5]

4.2.3. monochrome

[TBD]

4.2.4. prefers-color-scheme

[TBD]

4.2.5. prefers-reduced-motion

[TBD]

4.2.6. prefers-contrast

[TBD]

4.2.7. prefers-reduced-transparency

[TBD]

4.3. CSS Custom Properties

4.4. CSS Functions

4.5. Ishihara

5. Implementation

5.1. Preferences Weidget

5.2. Contrast Color Functions

5.3. Interactive Ishihara Plate

6. Discussion

7. Conclusions

Bibliography

[1] Florian Schnidrig, “Accessibility on the Web – Where We Stand.” [Online]. Available: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand

[2] Mozilla Corporation, “Using media queries.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using

[3] Mozilla Corporation, “forced-colors.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors

[4] Mozilla Corporation, “<system-color>.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/system-color

[5] Mozilla Corporation, “inverted-colors.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

List of Figures